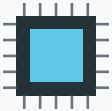
Computer Architecture • Chapter 4

Case Study: 32-bit and 64-bit Processors



IA-32, AMD64, registers, flat addressing, SSE/AVX, pipeline

Teacher: Dr. Benabderrezak Youcef

Course: UEF31, Architecture des ordinateurs

Goals of this chapter

At the end, you can:

- explain what changed from 16 to 32 and 64 bits;
- use the 64-bit registers and the base + index $\times$ scale addressing;
- explain SIMD instructions and the pipeline.

16, 32 or 64 bits?

	8086	IA-32	x86-64
register size	16	32	64
max memory	1 MB	4 GB	256 TB (48 bits)
general registers	8	8	**16**
names	AX	EAX	RAX, …, R15
addressing	segmented	flat	flat

An *n*-**bit** processor has *n*-bit general registers.

Why more than 32 bits?

1. **Memory**: 32-bit addresses stop at 4 GB.
2. **Performance**: big numbers in one instruction, and 16 registers.
3. **Compatibility**: old 32-bit programs still run.

4 GB exactly

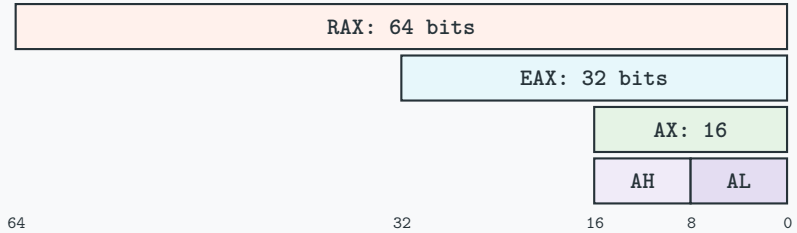
$2^{32} = 2^2 \times 2^{30} = 4$ GB and $2^{48} = 2^8 \times 2^{40} = 256$ TB.

From IA-32 to AMD64



Mode	Description
real	like an 8086, at start-up
protected	32 bits, memory protection
long	64 bits, plus a 32-bit compatibility sub-mode

Nested registers



New: **R8 to R15**, **RIP**, **RFLAGS**.
Parameters: RDI, RSI, RDX, RCX; result: RAX.

A test: what happens to the other bits?

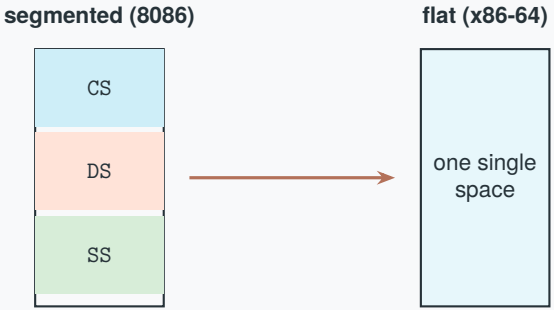
```
4  _start:
5      mov rax, 0x1122334455667788
6      call print_hex
7
8      mov rax, 0x1122334455667788
9      mov eax, 0xAABBCCDD
10     call print_hex
11
12     mov rax, 0x1122334455667788
13     mov ax, 0x1234
14     call print_hex
15
16     mov rax, 0x1122334455667788
17     mov al, 0xFF
18     call print_hex
```

```
1122334455667788
00000000AABBCCDD
1122334455661234
11223344556677FF
```

Rule

Writing a **32-bit** register clears the high 32 bits. Writing 8 or 16 bits keeps them.

From segments to a flat memory



In the **flat model**, an address is just one number in a 64-bit register.

The general form of an address

Rule

address $=$ base $+$ index $\times$ scale $+$ displacement (scale: 1, 2, 4 or 8)

NASM	Use
`[rbx]`	a pointer
`[rbx + 16]`	a field in a struct
`[tab + rcx*8]`	element i of an array of 8-byte numbers
`[rel msg]`	relative to RIP (64 bits only)

`[tab + rcx*8]` with RCX = 3 reads $3 \times 8 = 24$ bytes after `tab`.

Example: the sum of an array

```
4  _start:
5      xor rax, rax
6      xor rcx, rcx
7  again:
8      add rax, [tab + rcx*8]
9      inc rcx
10     cmp rcx, 5
11     jl  again
12     call print_dec
13
14     mov rax, 60
15     xor rdi, rdi
16     syscall
```

- Line 8: scaled index.
- Lines 9–11: `while (i < 5)`.

```
1030000014852
```

RIP-relative addressing

- `lea rsi, [rel buf]`: the address is RIP + distance.
- The program works **wherever** the system loads it.
- This helps security: the system moves programs at each run.

The same instruction on 16, 32 and 64 bits

Instruction	Machine code (real)	Note
`add ax, bx`	`66 01 D8`	prefix 66: 16 bits
`add eax, ebx`	`01 D8`	default size
`add rax, rbx`	`48 01 D8`	REX.W: 64 bits
`add r8, r9`	`4D 01 C8`	REX: new registers

01 D8 is the same since the 8086: the new sizes come from a **prefix**.

32 bits or 64 bits: hello

```
 8  _start:
 9      mov eax, 4
10      mov ebx, 1
11      mov ecx, msg
12      mov edx, len
13      int 0x80
14
15      mov eax, 1
16      mov ebx, 0
17      int 0x80
```

```
Hello from 32-bit mode!
```

```
 8  _start:
 9      mov rax, 1
10      mov rdi, 1
11      mov rsi, msg
12      mov rdx, len
13      syscall
14
15      mov rax, 60
16      mov rdi, 0
17      syscall
```

```
Hello from 64-bit mode!
```

32 bits: `int 0x80`, write = 4, parameters EBX, ECX, EDX. 64 bits: `syscall`, write = 1, parameters RDI, RSI, RDX.

What does the compiler write?

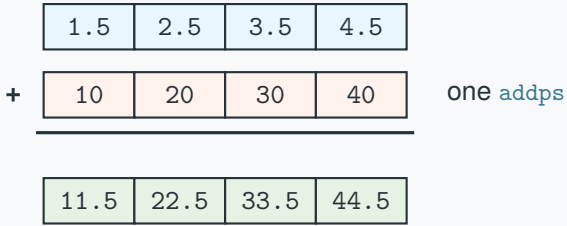
```c
/* compiled by gcc for x86-64 */
long sum(long *t, long n)
{
    long s = 0;
    for (long i = 0; i < n; i++)
        s += t[i];
    return s;
}
```

`gcc -O1 -S -masm=intel sum.c`

```
sum:
    test rsi, rsi
    jle .L4
    mov rax, rdi
    lea rcx, [rdi+rsi*8]
    mov edx, 0
.L3:
    add rdx, QWORD PTR [rax]
    add rax, 8
    cmp rax, rcx
    jne .L3
.L1:
    mov rax, rdx
    ret
.L4:
    mov edx, 0
    jmp .L1
```

`t` arrives in RDI, `n` in RSI; the result goes back in RAX.

SIMD: one instruction, several numbers



Extension	Registers	Size	floats at once
SSE	XMM	128 bits	4
AVX	YMM	256 bits	8
AVX-512	ZMM	512 bits	16

SSE in a real program

```
 5  int main(void)
 6  {
 7      float a[4] = {1.5, 2.5, 3.5, 4.5};
 8      float b[4] = {10, 20, 30, 40};
 9      float c[4];
10
11      __m128 x = _mm_loadu_ps(a);   /* load 4 floats into XMM */
12      __m128 y = _mm_loadu_ps(b);
13      __m128 z = _mm_add_ps(x, y);  /* ONE instruction: addps */
14      _mm_storeu_ps(c, z);
15
16      printf("%.1f %.1f %.1f %.1f\n", c[0], c[1], c[2], c[3]);
17      return 0;
18  }
```

```
11.5 22.5 33.5 44.5
```

In the compiled program:
`addps xmm0, [rbp-0x50]`

AVX version: `vaddps ymm0, …`

```
11 22 33 44 55 66 77 88
```

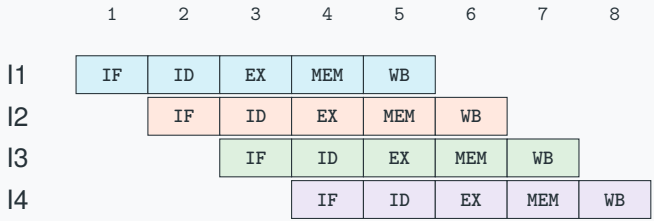
The five stages

Stage	Name	Work
IF	fetch	read the instruction
ID	decode	understand it, read registers
EX	execute	compute in the UAL
MEM	memory	read or write memory
WB	write back	store the result

Analogy

A laundry: wash, dry, iron. The second bag starts while the first one dries.

The pipeline



Cycles

without: $n \times k$ with: $k + (n - 1)$. 100 instructions, 5 stages: 500 vs 104 (4.8 times faster).

Hazards

- **Data**: `add rax, rbx` then `add rcx, rax`: forward the result, or wait.
- **Control**: after a jump, fetched instructions may be wrong: **branch prediction**.
- **Structural**: two instructions need the same circuit at once.

`mov`, `add`, `jmp` in the pipeline

	`mov rax, 60`	`add rax, rbx`	`jmp fin`
IF	48 C7 C0 3C 00 00 00	48 01 D8	EB 00
ID	64 bits, value in the instruction	read RAX and RBX	short jump
EX	–	RAX + RBX, flags	target = RIP + 0
WB	RAX $\leftarrow$ 60	RAX gets the result	RIP gets the target

A Core i7 has 14+ stages, starts 4 instructions per cycle, and runs them out of order.

Practice

Try it

1. RAX = FFFF…FFFF$_H$, then `mov eax, 1`: RAX? 0000000000000001$_H$
2. Address of element 10 of an `int` array at 601000$_H$? 601028$_H$
3. 1000 instructions, 6 stages: cycles with a pipeline? 1005

Summary

Remember

- x86-64: 16 registers of 64 bits, flat memory.
- address $=$ base $+$ index $\times$ scale $+$ displacement.
- SIMD: several numbers at once; pipeline: overlapping steps.